

Laboratorio 1: Tecnología De Microprocesamiento

David Alvarez
Ingeniería Mecatrónica
Universidad Tecnológica del Uruguay (UTEC)
Fray Bentos, Rio Negro, Uruguay
david.alvarez.s@estudiantes.utec.edu.uy

Gevont Utmazian
Ingeniería Mecatrónica
Universidad Tecnológica del Uruguay (UTEC)
Fray Bentos, Rio Negro, Uruguay
gevont.utmazian@estudiantes.utec.edu.uy

Abstract—Este informe presenta el desarrollo de un sistemas utilizando el microcontrolador ATmega328P, dividido en diferentes partes: una punzonadora con cinta transportadora, una matriz de LEDs con desplazamiento de mensajes , un DAC R-2R para generación de señales analógicas y un plotter. Todo fue programado íntegramente en lenguaje ensamblador. Se documentan los avances realizados, las decisiones de implementación y las etapas pendientes.

Index Terms—ATmega328P, Fischertechnik, UART, matriz de LEDs, DAC R-2R, ensamblador, mecatrónica.

I. INTRODUCCIÓN

El laboratorio tuvo como objetivo aplicar los conocimientos adquiridos sobre microprocesadores en un entorno práctico. Trabajamos con varios módulos físicos y de control, todos basados en el microcontrolador ATmega328P. La programación fue realizada en lenguaje ensamblador, lo que nos permitió tener un control más preciso del hardware y entender en profundidad cómo funciona a bajo nivel.

El trabajo se dividió en varias etapas: primero se armó la cinta transportadora con punzonadora del kit Fischertechnik; luego se implementó el control de una matriz de LEDs para visualizar mensajes por UART; posteriormente se generó una señal analógica mediante un DAC R-2R, y por último se planea integrar un sistema de trazado con un plotter controlado desde el micro.

II. OBJETIVOS

Los objetivos principales del laboratorio fueron:

- Controlar una cinta transportadora con punzonadora usando el ATmega328P.
- Programar una matriz de LEDs para mostrar mensajes y gráficos por UART.
- Implementar un DAC R-2R que genere señales analógicas desde datos digitales.
- Desarrollar el control de un plotter simple a través de comunicación serial.

Laboratorio 1 de la unidad curricular Tecnologías de Microprocesamiento,
Ingeniería Mecatrónica.

III. MARCO TEÓRICO

A. Microcontrolador ATmega328P

El ATmega328P es un microcontrolador de 8 bits de arquitectura AVR, ampliamente utilizado en aplicaciones de sistemas embebidos. Cuenta con 32 KB de memoria Flash, 1 KB de EEPROM y 2 KB de SRAM, además de múltiples puertos de entrada/salida digitales, timers, conversores analógico-digital (ADC), y periféricos de comunicación como UART, SPI e I2C.

En este laboratorio se utilizó para controlar todos los módulos: cinta transportadora, matriz de LEDs, DAC R-2R y el plotter. Programarlo directamente en ensamblador nos permitió trabajar a bajo nivel, manejando los registros internos, puertos de entrada/salida y temporizadores con precisión.

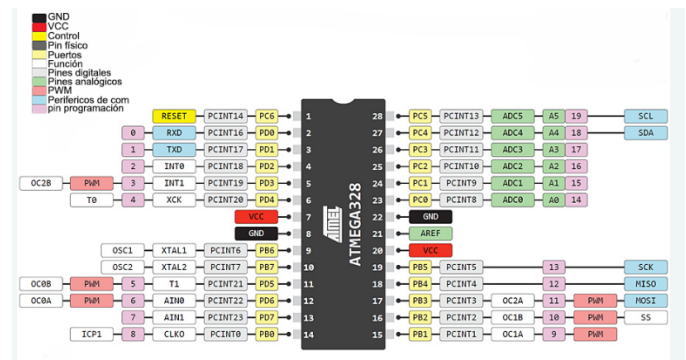


Fig. 1. Diagrama de pines del microcontrolador ATmega328P.

B. Comunicación UART

La UART (Universal Asynchronous Receiver/Transmitter) es un protocolo de comunicación serie asincrónica. En este laboratorio se usó para enviar comandos al microcontrolador y recibir respuestas o estados desde una terminal, sin necesidad de interfaces gráficas adicionales.

Fue fundamental para el control de los módulos, especialmente en el menú de la matriz de LEDs y en la futura selección de figuras a trazar con el plotter.

C. Matriz de LEDs 1088AS

La matriz de LEDs 1088AS es un componente de 64 LEDs organizados en una matriz 8x8. Permite representar carac-

terres, símbolos e imágenes simples mediante multiplexado por columnas y filas.

Para su funcionamiento, es necesario encender una fila a la vez y refrescar las columnas asociadas rápidamente, simulando que todos los LEDs están activos al mismo tiempo. En nuestro caso, utilizamos esta técnica para mostrar mensajes desplazables e imágenes predefinidas.

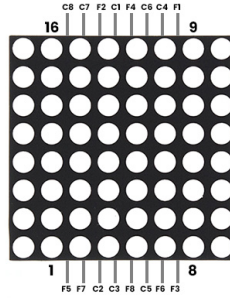


Fig. 2. Matriz de LEDs 1088AS y su disposición de pines.

D. Conversor DAC R-2R

El conversor R-2R es una red resistiva que convierte señales digitales en señales analógicas proporcionales. Se construye utilizando solo resistencias con valores R y 2R, y se conecta a las salidas digitales del microcontrolador.

En este laboratorio se usó para generar una señal analógica predefinida utilizando una tabla de valores (LUT) que se recorre secuencialmente. La señal resultante fue visualizada y verificada mediante un osciloscopio.

E. Máquinas de Estado

Una máquina de estados finita (FSM) es un modelo de control que permite estructurar sistemas secuenciales. Se usa para definir diferentes fases dentro de un proceso automatizado, como ocurre en la punzonadora: espera, alimentación, posicionamiento, punzonado, descarga y reinicio del ciclo.

F. Plotter

Un plotter es un sistema que traza líneas o figuras en papel u otra superficie. En nuestro caso, el modelo disponible en el laboratorio es controlado mediante señales digitales enviadas a un PLC, que a su vez acciona motores para mover el cabezal y una válvula neumática para activar el trazo.

El microcontrolador ATmega328P será el encargado de enviar estas señales de forma secuencial, según la figura seleccionada por el usuario a través de UART.

IV. METODOLOGÍA

Todo el código fue programado directamente en ensamblador para el ATmega328P. Esto nos permitió trabajar cerca del hardware, gestionar los registros directamente, y optimizar recursos.

A. Montaje inicial del sistema

Comenzamos con el armado del modelo físico de la cinta transportadora con punzonadora utilizando el kit Fischertechnik. Verificamos cada pieza con un checklist, leímos las instrucciones del manual (páginas 2 a 5) y seguimos el procedimiento paso a paso. No se utilizó el controlador del kit, ya que decidimos manejar todo desde el ATmega328P. El modelo quedó montado correctamente, listo para ser automatizado.

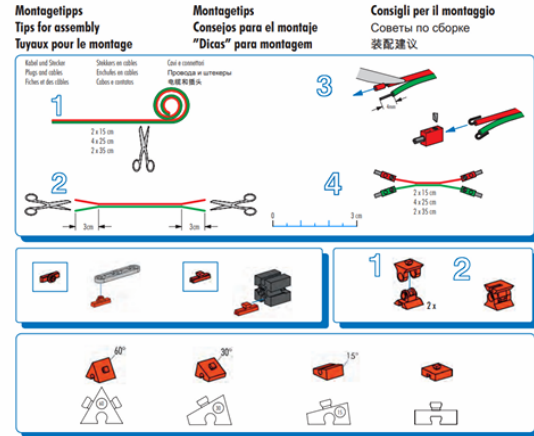


Fig. 3. Instrucciones básicas del kit del manual de montaje.

B. Parte 2A: Automatización de la punzonadora

En esta parte armamos una FSM (máquina de estados finita) para que la punzonadora con cinta trabaje sola. Programamos todo en ensamblador para el ATmega328P. Los estados que usamos son: ESPERA → CINTA_AVANCE → CINTA_PAUSA → POSICIONADO → PUNZON_BAJANDO → PUNZON_MANTENIENDO → PUNZON_SUBIENDO → DESCARGA → FIN → ESPERA.

a) Qué controla cada pin:

- Cinta (M2): PWM por OC0A (PD6/D6) y dirección por PB1 (D9).
- Punzón (M1): bajar con OC2B (PD3/D3) y subir con OC0B (PD5/D5).
- Botón inicio: PD2 con pull-up.
- LEDs: PORTC: PC0 (ligera), PC1 (media), PC2 (pesada), PC3 (espera), PC4 (funcionando), PC5 (fin).

Elegimos esos pines para no tener problemas con Timer1.

b) Timing y PWM (sin vueltas):

- Timer1 en CTC = 1 s por tick OCR1A=62499, prescaler 256. Lo usamos para contar segundos por fase.
- Timer0 Fast PWM (~976 Hz): OC0A mueve la cinta y OC0B sube el punzón.
- Timer2 Fast PWM (~976 Hz): OC2B baja el punzón.
- Rampas suaves: subimos/bajamos el duty en pasos (STEP_CINTA=10, STEP_PUNZON=15) hasta DUTY_CINTA=200 o DUTY_PUNZON=220. Para mantener abajo, usamos DUTY_MANTENER=80 (aprox. 30%).

c) *Cómo se maneja:* Por UART a 9600-8N1. Con '1'/'2'/'3' elegimos carga ligera/media/pesada (se prenden PC0/PC1/PC2) y con 'A' arrancamos el ciclo. También se puede empezar con el botón (PD2). En cada estado clave mandamos un texto por UART: "ESPERA", "ALIM", "POSICIONADO", "PUNZONADO", "DESCARGA", "FIN".

d) *Qué hace la ISR y el loop:* El *main loop* mira la variable estado y ejecuta lo que toca (cinta, punzón, LEDs, etc.). La `TIMER1_COMPA_ISR` descuenta `contador_timer` y, cuando llega a cero, pasa al siguiente estado. La `USART_RX_ISR` escucha 'A', '1', '2', '3' y actualiza estado/tipo_carga/LEDs.

Fase	Ligera	Media	Pesada
CINTA_AVANCE	3 s	4 s	5 s
CINTA_PAUSA	2 s	2 s	3 s
PUNZON_BAJANDO	1 s	1 s	1 s
PUNZON_MANTENIENDO	2 s	3 s	4 s
PUNZON_SUBIENDO	1 s	1 s	1 s
DESCARGA	3 s	4 s	5 s

e) *Tiempos por tipo de carga:*

f) *Cómo se ve el ciclo completo:* Desde ESPERA (PC3 encendido), con el botón o 'A' arrancamos. La cinta acelera suave (rampa de OCR0A) el tiempo que toque según la carga y luego frena suave. Avisamos POSICIONADO por UART. El punzón baja con rampa (OC2B), se queda abajo a ~30% el tiempo correspondiente, sube con rampa (OC0B) y vuelve a cero. Para descargar, la cinta vuelve a acelerar suave y, al terminar, apagamos todo con rampas a cero, cambiamos LEDs (fin) y volvemos a ESPERA.

g) *Código:* El ensamblador queda en el apéndice y en el repo de GitHub, con estos mismos pines, tiempos y rutinas.

C. Parte 2B: Matriz de LEDs y comunicación UART

En esta etapa trabajamos con una matriz de LEDs 1088AS conectada al microcontrolador ATmega328P montado sobre una placa Arduino UNO. Para proteger el hardware, se incorporaron resistencias que limitan la corriente en los pines de salida.

El código fue desarrollado completamente en ensamblador y permite visualizar el mensaje "WELCOME HOME" desplazándose de izquierda a derecha de manera continua y fluida. Además, implementamos un sistema de interacción mediante UART que puede ser controlado desde la terminal del entorno Microchip Studio.

Al iniciar el sistema, se presenta un menú que ofrece las siguientes opciones:

- Visualizar el mensaje desplazable.
- Mostrar una imagen de un corazón.
- Mostrar una carita feliz.
- Ajustar la velocidad de desplazamiento.

Cabe destacar que las figuras del corazón y la carita feliz ya habían sido desarrolladas previamente en una tarea de evaluación continua, por lo que fueron reutilizadas e integradas en este laboratorio sin necesidad de reprogramarlas desde cero.

Cada opción se activa enviando un carácter específico por la terminal. El sistema respondió correctamente en todas las pruebas realizadas, permitiendo cambiar entre modos de forma estable y en tiempo real.

D. Parte 2C: DAC R-2R y generación de señales

Los componentes principales utilizados para la generación de la señal analógica fueron:

- Microcontrolador ATmega328P.
- Conversor Digital-Analógico (DAC) de 8 bits, específicamente un R-2R Ladder implementado con resistencias de 3.3k.
- Osciloscopio Digital.
- Entorno de Desarrollo Atmel Studio y lenguaje Ensamblador AVR.

Se ensambló el circuito en una protoboard. Los ocho pines de datos del DAC se conectaron al Puerto D (PD2-PD7, B0-B1) del microcontrolador ATmega328P. La salida analógica del DAC se conectó a la entrada del Canal 1 del osciloscopio.

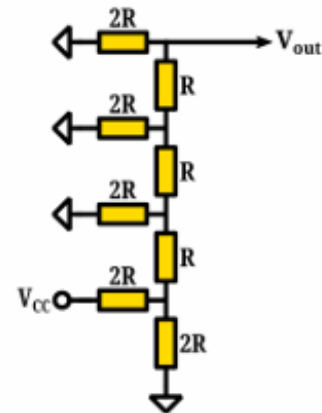


Fig. 4. Esquema DAC R-2R

El osciloscopio se utilizó para visualizar la señal generada y verificar sus parámetros. Se realizaron los siguientes ajustes: 2 V/div y 100ms.

E. Parte 2D: Control del plotter

Para llevar a cabo este proyecto, se utilizó un microcontrolador ATmega328P como unidad central de control del plotter. Las conexiones físicas entre el microcontrolador y los motores encargados del movimiento en los ejes X e Y fueron provistas por el docente en la consigna, lo que permitió enfocarse directamente en la programación del sistema. La comunicación entre el microcontrolador y el entorno externo se realizó mediante el protocolo USART, a través del cual se enviaron instrucciones codificadas que permitían mover el cabezal del plotter hacia arriba, abajo, izquierda y derecha, así como ejecutar comandos específicos para dibujar las figuras de la consigna.

La programación se desarrolló en lenguaje ensamblador. Se implementaron rutinas que interpretaban las instrucciones recibidas por USART y ejecutaban movimientos precisos para dibujar tres figuras: un triángulo, una cruz y un círculo. Mientras que el triángulo y la cruz se trazaron mediante trayectorias fijas definidas en el código, el círculo requirió una estrategia más elaborada debido a la naturaleza curva de su forma.

Para generar el círculo, se diseñó previamente una tabla de deltas de movimiento en Python. Estos deltas representaban desplazamientos relativos en los ejes X e Y, con valores entre 0 y 1, correspondientes al primer cuadrante del círculo (de 0° a 90°). Aprovechando la simetría de la figura, se almacenaron únicamente los deltas de ese cuadrante, reutilizándolos luego para los otros tres mediante transformaciones geométricas. Para optimizar el uso de memoria, cada delta fue codificado como un bit, y se agruparon ocho deltas por byte, generando así una tabla compacta de 8 bits por entrada. Esta tabla fue cargada en la memoria del microcontrolador y recorrida de forma iterativa para dibujar el círculo completo, repitiendo el mismo bloque de código cuatro veces, una por cada cuadrante.

V. RESULTADOS

A. Parte 2A: Punzonadora con cinta

Se automatizó la punzonadora con una FSM en ensamblador (ATmega328P) que ejecuta el ciclo completo y permite control/monitoreo por USART (9600-8N1). Desde ESPERA se inicia por botón o 'A', y '1'/'2'/'3' seleccionan la carga (ligera/media/pesada). Con Timer1 en CTC (1 s/tick) y PWM por Timer0/2 se obtuvieron transiciones estables sin bloqueos.

A nivel *lógico*, los tiempos se comportaron como fue programado (ver tabla en **Metodología**). Obviamente que hubieron variaciones de tiempos que pueden considerarse como "error" por causas mecánicas: holguras, rozamientos y, en particular, la cinta del kit que a veces se quedaba trabada o patinaba. Esto es esperable porque los FischerTechnik son *kits de prueba*, no un sistema industrial; por lo tanto, la sincronía perfecta entre tiempos lógicos y respuesta física no está garantizada. El control funcionó correctamente, pero ese pequeño desfase debe tenerse en cuenta al interpretar los resultados.

Por UART se observó la secuencia completa de estados en orden y el arranque remoto con 'A' respondió de forma consistente; los LEDs en PORTC reflejaron de inmediato la selección de modo.

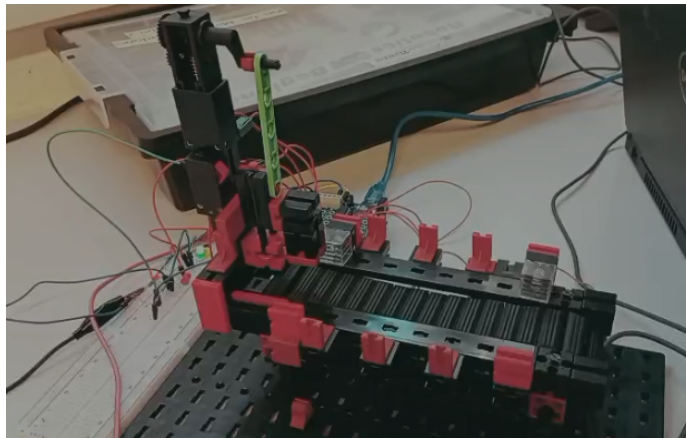


Fig. 5. Punzonadora en estado de ESPERA (LED PC3 encendido).

B. Parte 2B: Matriz de LEDs y UART

Se programó el ATmega328P para manejar la matriz 8×8 (DOLANG) y mostrar un mensaje desplazable con control por UART (9600-8N1). El firmware multiplexa filas/columnas con un displayBuffer de 8 bytes. El mensaje fue WELCOME HOME (14 caracteres), con desplazamiento de izquierda a derecha en nuestra implementación.

En pruebas, el scroll fue estable y legible a la velocidad configurada; la velocidad se ajusta por software (variable speed/retardos). Por UART se mostró un menú que respondió bien a: 1) scroll, 2) corazón, 3) carita, 4) ajuste de velocidad. Cabe recalcar que la luminosidad de la matriz no fue la mejor: el uso de protoboard (cables largos/tolerancias de resistencias) y la grabación con el celular a plena luz del día hicieron que algunos LEDs se vean más apagados que otros. Igual, el texto y las imágenes se leen bien en todas las pruebas. Con PCB o cableado más corto mejora. También mencionar que el cambio a desplazamiento derecha→izquierda es trivial (invertir el índice/orden de columnas en el loop).

C. Parte 2C: DAC R-2R y generación de señales

Una vez ensamblado el circuito en una placa perforada y cargado el programa en el microcontrolador ATmega328P, se procedió a verificar el funcionamiento del DAC R-2R de 8 bits. Los ocho pines de salida digital del microcontrolador fueron correctamente conectados al arreglo de resistencias R-2R, conformado por resistencias de 3.3kOhms, lo que permitió la conversión digital-analógica de los valores binarios enviados desde el puerto D y B.

La salida analógica del DAC fue conectada al Canal 1 del osciloscopio digital, configurado con una sensibilidad de 2 V/div y una escala temporal de 100 ms. Al ejecutar el programa, se observó en pantalla una señal periódica que correspondía a los niveles de tensión generados por el DAC en función de los valores digitales enviados. La forma de onda mostró una progresión ordenada de niveles, confirmando que el DAC estaba funcionando correctamente y que la conversión digital-analógica se realizaba de manera correcta.

La señal obtenida fue estable, sin presencia de ruido significativo pero con un par de fluctuaciones inesperadas, quizás generada por el DAC o quizás producto de copiar mal el LUT del archivo, faltando algunos valores, aunque se logro observar con claridad la onda esperada, lo que indica una buena calidad en el armado del circuito y en la implementación del código ensamblador. Además, se verificó que los niveles de tensión correspondían a los esperados para un DAC de 8 bits alimentado con una referencia de tensión adecuada, lo que permitió validar tanto el diseño del R-2R como la lógica de control implementada en el microcontrolador.

En conjunto, los resultados obtenidos demuestran que el sistema fue capaz de generar una señal analógica precisa a partir de datos digitales, utilizando una arquitectura sencilla y eficiente basada en resistencias discretas y control directo desde el microcontrolador.

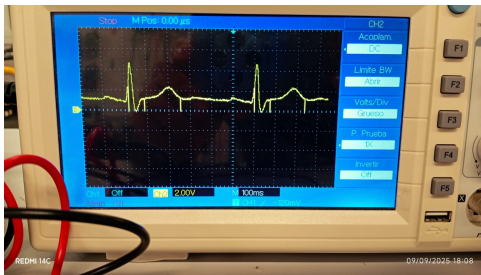


Fig. 6. Resultado del DAC

D. Parte 2D: Control del plotter

Una vez cargado el programa en el microcontrolador ATmega328p y establecida la comunicacion USART, se procedio a enviar las instrucciones correspondientes para verificar el funcionamiento del plotter. El sistema respondio correctamente a los comandos de movimiento en las cuatro direcciones, mostrando precision en los desplazamientos y estabilidad en la ejecucion.

Al ejecutar las instrucciones de dibujo, se observo que el triangulo y la cruz fueron trayectorias consistentes, sin tener en cuenta las perturbaciones generadas por la vibracion del lapiz o el movimiento de la hoja. La figura del triangulo mostro una correcta alineacion de sus vertices, mientras que la cruz presento simetria en sus brazos, confirmando su correcta interpretacion de las instrucciones.

El dibujo del circulo represento un desafio tecnico mayor, pero los resultados fueron satisfactorios. La tabla de deltas generada en Python permitio una interpolacion relativamente suave de la trayectoria de la curva. Al recorrer la LUT, en cuatro fases, correspondientes a los cuadrantes de un circulo, se logro una figura continua y simetrica. La compresion de los deltas en bytes de 8 bits no causo una degradacion de la calidad y genero una eficiencia en el codigo.

Finalmente, al enviar la instruccion compuesta que ordena el dibujo de las tres figuras en secuencia, el plotter ejecuto correctamente cada rutina, sin interrupciones ni errores. Esto afirma una correcta integracion de las figuras.

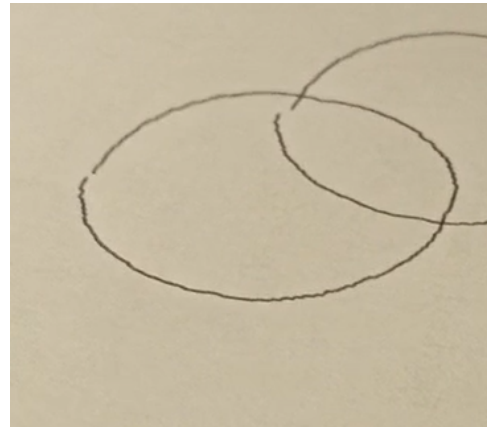


Fig. 7. Circulo dibujado por el plotter

VI. CONCLUSIONES

Comprobamos que todo corre en el ATmega328P en ensamblador: la FSM de la punzonadora fue estable, la matriz mostró mensajes/figuras por UART, el DAC R-2R generó la forma de onda prevista y el plotter dibujó triángulo, cruz y círculo con la LUT. Programar en ensamblador fue desafiante y obviamente trajo sus dificultades, pero nos dejó un control preciso del hardware. Toda la info útil (código, fotos y videos) están en el repositorio de GitHub y en el Drive del Apéndice.

APÉNDICE

Todo el código y archivos de este laboratorio están disponibles en el siguiente repositorio público:

- Repositorio GitHub: <https://github.com/gevontutmazian8/Tecnologias-de-microprocesamiento.git>

Además, para mostrar el funcionamiento real de cada módulo, grabamos videos con las pruebas realizadas. Están organizados en una carpeta de Google Drive accesible desde este enlace:

- Videos de funcionamiento: <https://drive.google.com/drive/folders/1EFU-1KBUDHmB6uMWeI3T3a1XvOhzlHex?usp=sharing>

REFERENCES

- [1] ATmega328P Datasheet, Microchip Technology Inc.
- [2] Fischertechnik – Manual de montaje industrial.
- [3] Ficha técnica de la matriz de LEDs 1088AS.
- [4] Apuntes del curso Tecnologías de Microprocesamiento, Semestre IV.